Universitè Grenoble Alpes - INP

Master of Science in Industrial and Applied Mathematics

Computational Biology

Salmonella Outbreak

Alibrandi Orazio, Guarnaschelli Eleonora

Academic year: 2025-2026

Indice

1 Introduction **3**

2 Mathematical Justification **4**
2.1 Data filtering using k-mers. 4
2.2 Search for the SNP 6

3 Python Implementation **7**

4 Results **11**

5 Drawbacks and Future work **13**

Riferimenti bibliografici **14**

1 Introduction

Problem Understanding Antimicrobial resistance (AMR) occurs when microorganisms evolve mechanisms that protect them from antimicrobials, drugs used to treat infections. A major subset of AMR is antibiotic resistance, which allows bacteria to survive antibiotic treatment and complicates infection management. This resistance can emerge through spontaneous mutations, horizontal gene transfer or selective pressure from antibiotic overuse in medicine and agriculture.

Antibiotic resistance is a property of the microorganism rather than the host, and can be classified as microbiological or clinical. The former results from genetic mutations or inherited genes that neutralize antibiotic action, while the latter is observed when bacteria which are normally susceptible become resistant after exposure. Understanding these genetic mechanisms is crucial to identify the origin of resistance and to develop effective countermeasures.

Salmonella is a bacterial species that can develop resistance to antibiotics through genetic mutations. In this project, we investigate a mutated strain of Salmonella that has acquired antibiotic resistance and the main goal is to identify and characterize the mutation responsible for this phenomena. In particular, we study two strains of the same species: a wild-type strain (WT), which is non-resistant, and a mutated strain (R) that has acquired resistance to tetracycline. Another important result to achieve is the development of a linear algorithm that identifies the mutation and then provide its biological interpretation.

Problem Specification We are provided with two $FASTA$ files containing N_r sequencing reads of 250 bp each derived from the Salmonella Bacteria genome. The first file corresponds to the WT genome and the second to the resistant strain R. The two genomes differ due to a single mutation, most likely a single-nucleotide polymorphism (SNP). A SNP is a substitution of a single nucleotide at a specific position in the genome, and its distribution is not homogeneous: SNPs occur more frequently in non-coding regions and their density can be influenced by factors such as mutation rate, genetic recombination, and local sequence composition, including the presence of microsatellites.

We start from the following assumptions:

1. Reads overlap along the genome, making the concept of coverage, i.e. the number of reads covering a specific genomic position, fundamental for our analysis.

2. Reads are uniformly distributed along the genome.

3. The sequencing process introduces random errors with probability $\nu \approx 0.01$, corresponding to approximately 2.5 sequencing errors per read.

4. There is a single mutation differentiating the two strains, but it may appear in two positions due to the reverse complementarity of DNA strands.

To achieve our objective, we will compare the WT and R datasets to locate the mutated region and extract the genomic segment carrying the mutation. Specifically, we aim to isolate the read surrounding the mutation, which can then be used as input for the online tool **BLASTX** to perform a biological translation of the mutation in the corresponding protein and understand its potential functional impact.

2 Mathematical Justification

In this section, we provide a mathematical description and justification of the statistical model underlying our method. Our workflow consists of two main phases:

(i) *data filtering using k-mers*

(ii) *search for the SNP.*

2.1 Data filtering using k-mers.

As previously discussed, sequencing reads are affected by random errors. Since our goal is to identify the mutation responsible for antibiotic resistance, an initial data-cleaning step is required to remove sequencing artifacts. We exploit k-mers and their count distribution (the k-mer spectrum) for this purpose. In a genomic context, a k-mer is a substring of length k contained within a read. A sequence of length L contains $L - k + 1$ k-mers, and the total number of distinct k-mers over the nucleotide alphabet is 4^k. The k-mer spectrum (or k-mer histogram) shows the multiplicity of each k-mer in a sequence versus the number of k-mers with that multiplicity. It provides insight into genome complexity and allows distinguishing sequencing errors, as erroneous k-mers typically occur much less frequently than true ones.

Our filtering strategy relies on analyzing this spectrum to separate solid k-mers (those originating from the true genome) from weak k-mers (likely containing sequencing errors). The first step is finding the k-mers and then computing the occurrence of every k-mer in the read set. Under the assumptions introduced in the previous section, the count of a specific solid k-mer across all reads can be approximated as a Binomial random variable:

$$X_k \sim \text{Binomial}\big(N_r,\ p(1-\nu)^k\big),$$

where N_r is the total number of reads, ν is the sequencing error rate, and $p \approx (L-k)/(G-L)$ is the probability that a read overlaps the k-mer, given read length L and genome length G.

Conversely, the count of an erroneous k-mer that differs from a true one by a single sequencing error can be approximated by:

$$Y_k \sim \text{Binomial}\big(N_r,\ p\,\nu(1-\nu)^{k-1}\big),$$

assuming that only one error occurs within the k-mer and all other positions are correct, since the probability to have two errors in such a small piece of read is negligible.
It is important to notice that this theoretical model is not meant to fit real data precisely, but rather to provide intuition on how sequencing errors shape the k-mer spectrum and to guide us for the choice of k.

Choice of k and anchoring position. The parameter k controls two key aspects:

(i) *how uniquely a k-mer anchors to the genome*

(ii) *how robust it is to sequencing errors*

If k is too small, many reads share identical k-mers, reducing uniqueness. If k is too large, even solid k-mers become rare because the probability of being error-free, $(1-\nu)^k$, decreases exponentially.

Thus, the choice of k is a trade-off between uniqueness and error tolerance. A practical lower bound can be estimated from the expected number of unique k-mers in a genome of length G, i.e., $G/4^k$, leading to $k \gtrsim \log_4 G$. In our case ($G \approx 5^6$): this gives us an estimate of $k \geq 11$. In practice,

short-read data sets benefit from choosing k in the range 21–31, which yields mostly unique anchors while maintaining sufficient coverage.

Empirically, the k-mer histogram shows two distinct modes: a low-frequency peak corresponding to erroneous k-mers (Y_k) and a high-frequency peak corresponding to solid ones (X_k). The optimal k maximizes the separation between these two distributions. From a computational perspective, values $k \leq 32$ are convenient, as they fit within 64-bit integers and allow fast comparisons. Finally, odd k values are often preferred so that forward and reverse complement k-mers are distinct. As presented in the article [1], the largest odd k that is computationally efficient is 31, which makes it a popular choice for a wide range of applications, and it is the one we selected to address this situation.

Threshold selection. Once the value of k is fixed to ensure a unique anchoring of k-mers, the next step in the filtering procedure is to define a threshold f_0 that distinguishes solid from erroneous k-mers. In theory, the frequency distributions of solid and erroneous k-mers, denoted respectively as $P_{\text{solid}}(f)$ and $P_{\text{error}}(f)$, are not completely disjoint but overlap in a small region where classification becomes uncertain.

To solve this issue, we should introduce two thresholds: a lower bound f_0^-, obtained by testing the tail of $P_{\text{error}}(f)$ to reject false positives, and an upper bound f_0^+, estimated from the left tail of $P_{\text{solid}}(f)$ to reject false negatives. k-mers with frequency $f < f_0^-$ are classified as erroneous, while those with $f > f_0^+$ are considered solid. The k-mers lying in the intermediate region $f_0^- < f < f_0^+$ fall within the intersection of the two distributions and remain unclassified.

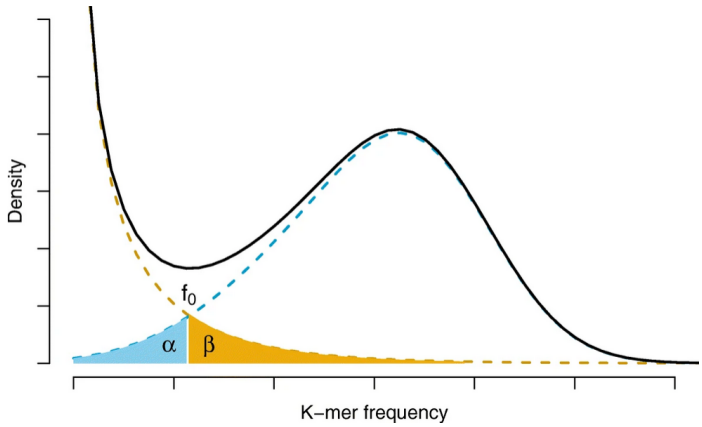

Figura 2.1: Visualization of threshold selection. The area labeled by α corresponds to the proportion of solid k-mers misclassified as erroneous ($f < f_0$), and the area labeled by β represents the proportion of erroneous k-mers misclassified as solid ($f > f_0$).

In an ideal case, when k is large enough to ensure a clear separation between the two populations, the overlap area becomes negligible, leading to $f_0^- \approx f_0^+ = f_0$. This unique threshold f_0 minimizes the overall misclassification rate

$$E(f_0) = \alpha(f_0) + \beta(f_0),$$

and can be estimated either visually, by locating the intersection of the two components, or mathematically, by fitting a two-component model to the observed k-mer spectrum and finding the point that separates the two groups.

In our setting, rather than performing a full statistical fit, we adopted a practical and interpretable approach based on visual inspection of the empirical k-mer histogram in logarithmic scale (Figure 2.2). From 2.2, it is evident that a local minimum exists around a coverage of 13, separating the low-frequency error peak from the higher-frequency solid peak. To ensure that ambiguous k-mers in

the overlap region were safely excluded, we chose a slightly higher cutoff, setting $f_0 = 17.5$. This conservative choice minimizes the probability of including erroneous k-mers, while retaining nearly all true ones. Given the observed separation between the two distributions and the chosen value of k, it is statistically very unlikely that a true mutation falls within the narrow overlap region.

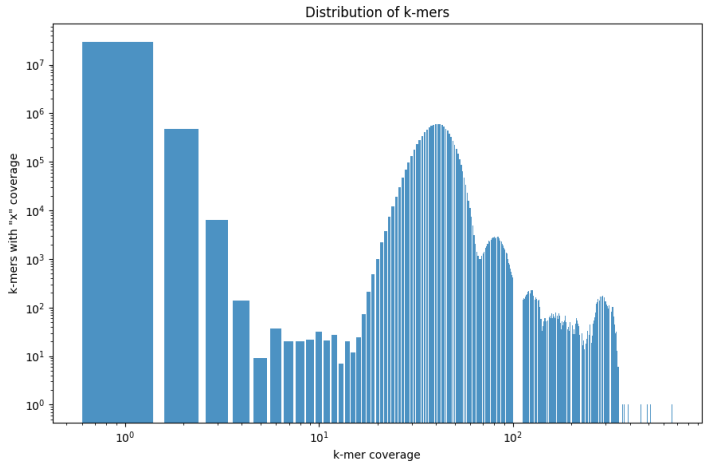

Figura 2.2: Empirical 31-mer histogram in logarithmic scale used for visual threshold selection. The local minimum around coverage 13 represents the boundary between the erroneous and solid k-mers. The final threshold was set to $f_0 = 17.5$.

Finally, only k-mers with counts $c \geq f_0$ were classified as solid for further analyses.

2.2 Search for the SNP

The second phase consist in the use of the threshold to obtain the solid k-mers from both datasets. In fact, all k-mers with observed frequency below c are discarded, producing a cleaned k-mer dataset composed mostly by solid k-mers. Two sets were thus obtained: $\mathcal{X}_k^{(wt)}$ for the wild-type and $\mathcal{X}_k^{(mut)}$ for the mutated genome. The intersection between these two sets,

$$\mathcal{I}_k = \mathcal{X}_k^{(wt)} \cap \mathcal{X}_k^{(mut)},$$

represents all k-mers common to both genomes and therefore unaffected by the mutation. Conversely, the elements in the complements $\mathcal{X}_k^{(wt)} \setminus \mathcal{I}_k$ and $\mathcal{X}_k^{(mut)} \setminus \mathcal{I}_k$ correspond to k-mers unique to each version of the genome and are potential candidates containing the mutation.

To efficiently identify pairs of k-mers differing by exactly one nucleotide, a mask-based search algorithm was implemented. For each k-mer, all possible masked forms obtained by replacing one position with a wildcard symbol ($*$) were indexed. Then, each k-mer from the mutated set was compared to the masked index of the wild-type set, and pairs differing by a single base were recorded:

$$\text{pairs} = \{(x, y) \in \mathcal{X}_k^{(wt)} \times \mathcal{X}_k^{(mut)} \mid d_H(x, y) = 1\},$$

where $d_H(\cdot, \cdot)$ denotes the Hamming distance. This procedure efficiently detects local single-base substitutions (SNPs).

Finally, each k-mer pair was mapped back to the corresponding read in the mutated FASTA file to reconstruct the sequence context in which the substitution occurs. This allowed the precise localization of the mutation within the original read, confirming the consistency of the detected difference at the sequence level.

3 Python Implementation

In this section we present the Python implementation of the whole reasoning done in the previous part.

```python
# %%
import collections
from collections import Counter
from Bio import SeqIO
import matplotlib.pyplot as plt
from scipy.stats import binom
```

Listing 1: Importing libraries

```python
# %%

# Get k-mers and their coverage
def count_fasta_kmers(filename, k):
    counts=collections.defaultdict(int) # Creates empty dict
    N_r=0
    for record in SeqIO.parse(filename, "fasta"):
        read=str(record.seq)
        N_r+=1
        num_kmers=len(read)-k+1
        for i in range(num_kmers):
            kmer=read[i:i+k]
            counts[kmer]+=1
    return counts, N_r
```

Listing 2: Counting k-mers from a FASTA file

```python
def get_axes(counts):
    kmer_counts=list(counts.values()) # Extract coverage of each k-mer

    # Counts how many k-mers have a certain coverage
    freq_of_counts=Counter(kmer_counts) # Creates a dictionary where keys=
        counts.values

    x=sorted(freq_of_counts.keys())
    y=[freq_of_counts[i] for i in x] # List of sorted values

    return x, y
```

Listing 3: Computing axes for k-mer coverage distribution

```python
def plot_kmer_frequency_distribution(x,  y, logx=False, logy=False):

    plt.figure(figsize=(9,6))
    plt.bar(x, y, width=0.8, align='center', alpha=0.8) # Creates hist

    plt.xlabel("k-mer coverage")
    plt.ylabel("k-mers with \"x\" coverage")
    plt.title("Distribution of k-mers per coverage")

    if logx:
        plt.xscale('log')
    if logy:
        plt.yscale('log')
```

```

    plt.tight_layout()
    plt.show()
```

Listing 4: Plotting k-mer frequency distribution

```
def plot_kmer_with_theoretical(x, y, N_r, pX_k, pY_k):

    pmfX = binom.pmf(x, N_r, pX_k)
    pmfY = binom.pmf(x, N_r, pY_k)

    plt.figure(figsize=(9,6))
    plt.plot(x, pmfX, 'r-', lw=2, label="Binomial X_k")
    plt.plot(x, pmfY, 'g--', lw=2, label="Binomial Y_k")
    plt.xlabel("k-mer coverage")
    plt.ylabel("Probability")
    plt.title("Theoretical binomial distributions")
    plt.legend()
    plt.tight_layout()
    plt.show()
```

Listing 5: Plotting k-mer theoretical distributions

```
# %%
# File paths
fasta_file_wt="salmonella-enterica.reads.fna"
fasta_file_m="salmonella-enterica-variant.reads.fna"
```

Listing 6: Defining file paths for input data

```
# %%
# Fix the length of the k-mer
k=35

# Sequencing error
nu=0.01

# Fix the genome size
genome_size= 5e6
read_length=250

# Probability for a k-mers to overlap a reads
p=(read_length-k)/(genome_size-read_length)

# Compute the empirical probabilities for X_k and Y_K
pX_k=p*(1-nu)**k
pY_k =p*nu*(1-nu)**(k-1)
```

Listing 7: Setting sequencing parameters and theoretical probabilities

```
# Build dataset of k-mers and their coverage and take the number of reads
counts_wt, N_r=count_fasta_kmers(fasta_file_wt, k)
x, y=get_axes(counts_wt)

# Hist of empirical distribution
plot_kmer_frequency_distribution(x, y, False, False)
```

```python

# Plot of theoretical distributions
plot_kmer_with_theoretical(x, y, N_r, pX_k, pY_k)
```

Listing 8: Building k-mer dataset and plotting empirical and theoretical distributions

```python
# %%
# Extract solid k-mers from the wild-type k-mers

threshold=17.5

solid_kmers_wt=[]
for key, v in counts_wt.items():
    if v > threshold:
        solid_kmers_wt.append(key)
```

Listing 9: Extracting solid k-mers based on coverage threshold

```python
# %%
# Do the same with mutated one

# Build dataset of k-mers and their coverage and take the number of reads
counts_m, _=count_fasta_kmers(fasta_file_m, k)

solid_kmers_m = []
for key, v in counts_m.items():
    if v > threshold:
        solid_kmers_m.append(key)
```

Listing 10: Processing the mutated sample in the same way

```python
# %%
# Find k-mers with mutation

# Find intersection
intersection = set(solid_kmers_wt) & set(solid_kmers_m)
not_matched_wt = set(solid_kmers_wt) - intersection
not_matched_m  = set(solid_kmers_m)  - intersection
```

Listing 11: Finding k-mers corresponding to mutations

```python
# %%
# Hamming-Neighbourhood algorithm in order to find the mutation

def build_mask_index(kmer_list):
    index = {}
    for km in kmer_list:
        for i in range(len(km)):
            mask = km[:i] + '*' + km[i+1:]
            if mask not in index:
                index[mask] = []
            index[mask].append(km)
    return index
```

```python
def find_one_base_diff(kmers_a, kmers_b):
    index_a = build_mask_index(kmers_a)
    pairs = []
    for km in kmers_b:
        for i in range(len(km)):
            mask = km[:i] + '*' + km[i+1:]
            if mask in index_a:
                for match in index_a[mask]:
                    if sum(aa != bb for aa, bb in zip(km, match)) == 1:
                        pairs.append((match, km))
    return pairs
```

Listing 12: Hamming-neighbourhood algorithm to detect one-base differences

```python
# %%
# Find and print mutations
pairs = find_one_base_diff(not_matched_wt, not_matched_m)
print(f"{len(pairs)} k-mers differs by one basis.")
print(pairs) #len(pairs) #4      #len(pairs[0]) #2
```

Listing 13: Finding and printing the mutation pairs

```python
# %%
# Recover bigger sequence (read)

sequences=[]

for i in range(len(pairs)):
    for j in SeqIO.parse(fasta_file_m, "fasta"):
        read=str(j.seq)
        if pairs[i][1] in read:
            sequences.append(read)
            break

print(sequences)
```

Listing 14: Recovering larger sequences (reads) containing the mutation

4 Results

After running the implemented code, two main outputs were obtained, corresponding respectively to the identification of mutated k-mers and to the extraction of the original reads in which these mutations occur.

Identification of mutated k-mers. The first step of the analysis produced the following message: **"4 k-mers differ by one basis."** This means that four pairs of k-mers were found to differ from each other by a single nucleotide, suggesting the presence of potential point mutations. These pairs are reported in Table 1.

Tabella 1: Pairs of k-mers differing by one nucleotide.

```
(TACGGGTTGTTAAACCTTCGATTCCGACCTCATTA, GACGGGTTGTTAAACCTTCGATTCCGACCTCATTA)
(ATGTAGGCTGCTCTACACCTAGCTTCTGGGCGAGT, ATGTAGGCTGCTCTACACCTAGCTTCTGGGCGAGG)
(ACTCGCCCAGAAGCTAGGTGTAGAGCAGCCTACAT, CCTCGCCCAGAAGCTAGGTGTAGAGCAGCCTACAT)
(TAATGAGGTCGGAATCGAAGGTTTAACAACCCGTA, TAATGAGGTCGGAATCGAAGGTTTAACAACCCGTC)
```

Each pair of sequences differs by only one character, thus pinpointing the precise location of the mutation at the nucleotide level. From these results, we were able to identify the specific k-mers where the mutation resides.

Retrieval of full reads containing the mutation. To further investigate the biological significance of this mutation, we retrieved the original reads from the mutated dataset containing the identified k-mers. The complete sequences are reported below:

```
AAAATTAGGAATTAATGATGTCTAGATTAGATAAAAGTAAAGTGATTAACAGCGCATTAGAGCTGCTTAATGAGGTCGGAATCGAAGGTTTAAC
AACCCGTCCCCTCGCCCAGAAGCTAGGTGTAGAGCAGCCTACATTGTATTGGCATGTAAAAAATAAGCGGGCTTTGCTCGACGCCTTAGCCATT
GAGATGTTAGATAGGCACCATACTCACTTTTGCCCTTTAGAAGGGGAAAGCTGGCAAGATTT

AATCTTGCCAGCTTTCCCCTTCTAAAGGGCAAAAGTGAGTATGGTGCCTATCTAACATCTCAATGGCTAAGGCGTCGAGCAAAGCCCGCTTATTT
TTTACATGCCAATACAATGTAGGCTGCTCTACACCTAGCTTCTGGGCGAGGGGACGGGTTGTTAAACCTTCGATTCCGACCTCATTAAGCAGCTCT
AATGCGCTGTTAATCACTTTACTTTTATCTAATCTAGACATCATTAATTCCTAATTTTT

AATCTTGCCAGCTTTCCCCTTCTAAAGGGCAAAAGTGAGTATGGTGCCTATCTAACATCTCAATGGCTAAGGCGTCGAGCAAAGCCCGCTTATTT
TTTACATGCCAATACAATGTAGGCTGCTCTACACCTAGCTTCTGGGCGAGGGGACGGGTTGTTAAACCTTCGATTCCGACCTCATTAAGCAGCTCT
AATGCGCTGTTAATCACTTTACTTTTATCTAATCTAGACATCATTAATTCCTAATTTTT

AAAATTAGGAATTAATGATGTCTAGATTAGATAAAAGTAAAGTGATTAACAGCGCATTAGAGCTGCTTAATGAGGTCGGAATCGAAGGTTTAAC
AACCCGTCCCCTCGCCCAGAAGCTAGGTGTAGAGCAGCCTACATTGTATTGGCATGTAAAAAATAAGCGGGCTTTGCTCGACGCCTTAGCCATT
GAGATGTTAGATAGGCACCATACTCACTTTTGCCCTTTAGAAGGGGAAAGCTGGCAAGATTT
```

These sequences were then used as input for **BLASTX**, a tool that translates nucleotide sequences into all six possible reading frames and compares the resulting protein sequences against a protein database.

Protein identification. The **BLASTX** analysis revealed that the mutation is associated with the following amino acid sequence:

```
MMSRLDKSKVINSALELLNEVGIEGLTTRPLAQKLGVEQPTLYW
HVKNKRALLDALAIEMMMSRLDKSKVINSALELLGIEGLTTRLA
QKLGVEQPTLYWHVKNKRALLDALAIEMMMSRLDKSKVINSALE
LLNEVGIEGLTTRKLAQKLGVEQPTLYWHVKNKRALLDALAIEM
```

Finally, to determine the biological role of the corresponding protein, the translated sequence was submitted to **HHMER** for functional annotation. The output identified the protein as belonging to the *TetR family transcriptional regulators*:

Protein name: TetR family transcriptional regulator

> **Function:** "TetR is the repressor of the tetracycline resistance element; its N-terminal region forms a helix-turn-helix structure and binds DNA. Binding of tetracycline to TetR reduces the repressor affinity for the tetracycline resistance gene (*tetA*) promoter operator sites."

This finding suggests that the detected mutation occurs in a genomic region related to the *TetR* regulatory system, which is known to control genes conferring resistance to tetracycline antibiotics. Therefore, the mutation likely contributes to the modulation of tetracycline resistance, potentially enhancing bacterial survival in the presence of the drug.

5 Drawbacks and Future work

The approach proposed in this report leaves room for several future improvements, which include the use of mathematical and statistical tools to increase the rigor of the solution. Referring to the approach proposed in the paper [2], a statistical model that actually fits the real data can be build, for example using Bayesian Statistics to derive the parameters of the distributions of solid and erroneus k-mers (which can be a mixture of binomial, poisson or gaussian distribustrions), given that the genome length is not strictly 5Mbp or that the error rate may be slightly different from 1 %.

It is evident if we plot the theoretical distributuions we assumed compared with the histogram obtained from data:

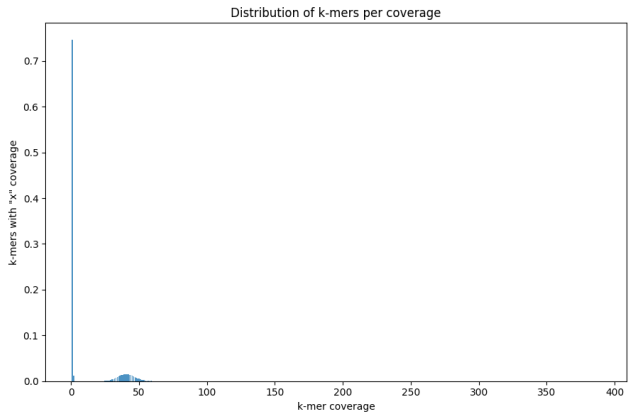

Figura 5.1: 31-mers empirical distribution

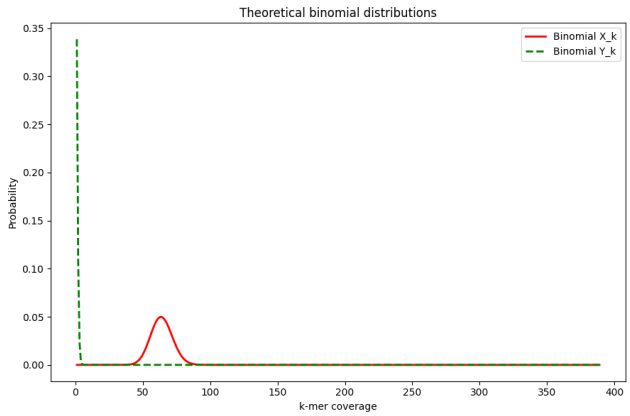

Figura 5.2: 31-mers theoretical distributuions.

We assume that the coverage frequencies of solid and erroneous k-mers follow two distinct Binomial distributions, forming a mixture model. Specifically,

$$f_x \sim \text{Binomial}(N_r, p_x), \quad f_y \sim \text{Binomial}(N_r, p_y),$$

Thus, the overall likelihood can be expressed as a mixture:

$$L(f \mid \theta) = \pi \, \text{Bin}(f \mid N_r, p_x) + (1 - \pi) \, \text{Bin}(f \mid N_r, p_y),$$

where π denotes the proportion of solid k-mers.

A Bayesian estimation of parameters $\theta = \{p_x, p_y, \pi\}$ can be performed using MCMC (e.g., with `Stan`) or via the Expectation-Maximization (EM) algorithm. Based on the estimated posterior distributions of solid and erroneous k-mers, we can determine an optimal frequency threshold f_0 that minimizes the misclassification areas α and β:

$$\alpha = P(f < f_0 \mid \text{solid}), \quad \beta = P(f > f_0 \mid \text{erroneous}).$$

The optimal threshold f_0^* can be obtained as:

$$f_0^* = \arg\min_{f_0} \left[\alpha(f_0) + \beta(f_0) \right].$$

Riferimenti bibliografici

[1] Katharine M. Jenike, Lucía Campos-Domínguez, Marilou Boddé, José Cerca, Christina N. Hodson, Michael C. Schatz, and Kamil S. Jaron. k-mer approaches for biodiversity genomics. *Genome Research*, 35(2):219–230, 2025.

[2] Liang Zhao, Jin Xie, Lin Bai, Wen Chen, Mingju Wang, Zhonglei Zhang, Yiqi Wang, Zhe Zhao, and Jinyan Li. Mining statistically-solid k-mers for accurate ngs error correction. *BMC Genomics*, 19(Suppl 10):912, 2018.